Scurry Protocol Specification
SCURRY 1.2

Validated design revision: **2026-10-23**. Illustrated revision: topology and numbered exchange diagrams added in the original document's box-and-arrow style. This revision corrects the uploaded architecture specification and incorporates the detailed 1.2 design. It supersedes the HTTP relay and server-to-server trunk illustrations in that PDF.

Implementation revision: 2026-10-16. Source release: **ScurryMsg 1.11.0**. The 1.2 baseline implementation revision remains 2026-10-16; the new binary input is added 2026-10-23. The owner explicitly requested implementation of 1.2 and deprecation of 1.0 in the project instructions. This supersedes the earlier proposed-only status of the 1.2 checksum. It does not claim outside standards-body approval or approve the separate 1.1 resource proposal.

Validation and corrections

The supplied three-page architecture PDF was reviewed in full against the requested 1.2 flows and the available ScurryMsg 1.10.0 source. It needed substantive corrections. The following table records the changes; the sections below define the corrected design. This is a document review and source cross-check, not a fresh runtime certification.

Original description	Corrected design
HTTP POST and JSON carry peer messages	SCURRY fixed-slot binary frames over verified TLS/TCP carry messages. Some endpoint bodies contain JSON; that does not make the transport HTTP.
Webserver HTTP lookup is required	The existing contact directory uses a separate UDP JSON exchange keyed by full name and email. Manual contact configuration is also possible. The HTTP lookup illustration is not an implemented 1.2 endpoint.
Persistent central relay pushes to Client B	Central mode stores mail in server mailboxes. The receiving client retrieves mail and acknowledges delivery; a permanent push connection is not required.
Server A sends payload through an S2S trunk to Server B	The receiving client follows a token notice, downloads from Server A, and imports the full original message into its own Server B. No automatic S2S trunk is specified.
SHA-256 called a signature	SHA-256 verifies payload consistency; the deterministic SHA-512 receipt is also a checksum, not a digital signature.
Guaranteed firewall traversal and relative latency/privacy ratings	Connectivity, latency and privacy depend on reachability, deployment and trust. Central mode uses outbound client connections but cannot guarantee firewall passage.
No version, receipt or authentication contract	Require the first Protocol record, canonical credential names, binary field lengths, same-connection peer receipts, central receipt consumption and retry rules below.

Operational topology

Peer-to-peer

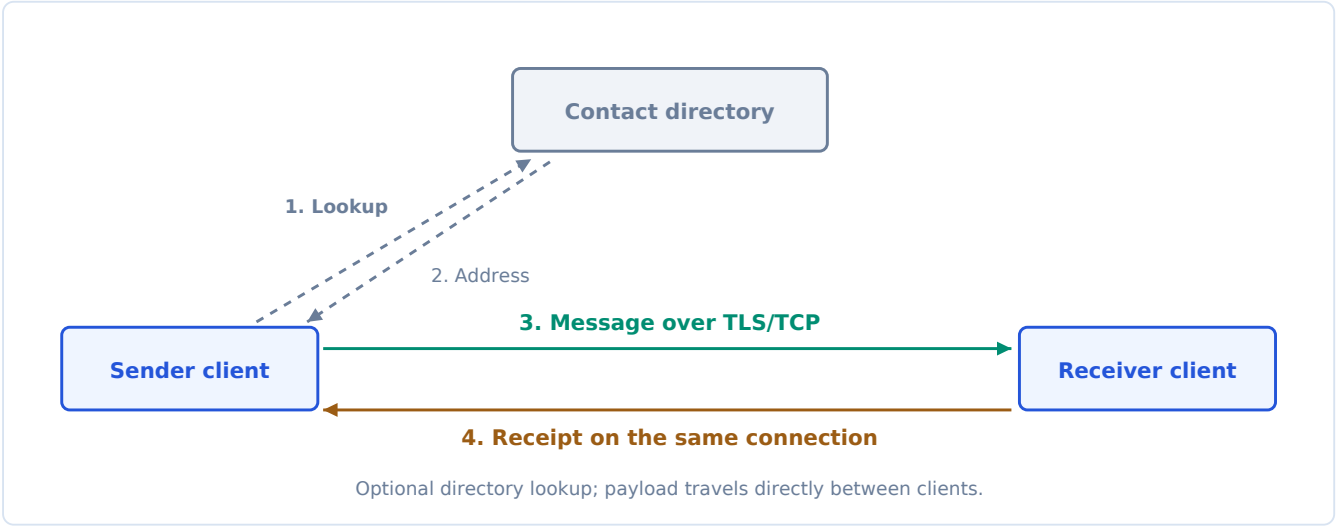


Figure 1. Direct delivery with optional lookup. The directory resolves an address only. The receiver returns the receipt after verified, durable completion, including resumed completion. A completed retry replays the saved receipt on that retry connection.

The sender contacts the receiver directly over verified TLS/TCP. A separate directory can resolve a contact; it does not relay the message and does not replace TLS/contact authentication. Package default ports are UDP 33336 for directory discovery and TCP 33337 for peers; these are configurable conventions, not assigned standard ports. A directory lookup result does not prove identity or external reachability.

The sender sends `/message` or another supported peer content endpoint, then reads `/delivery-success` on the same connection after completion. NAT may require explicit inbound reachability or port forwarding. The design makes no claim that STUN, UPnP or automatic mode fallback is implemented.

Central mailbox on one server

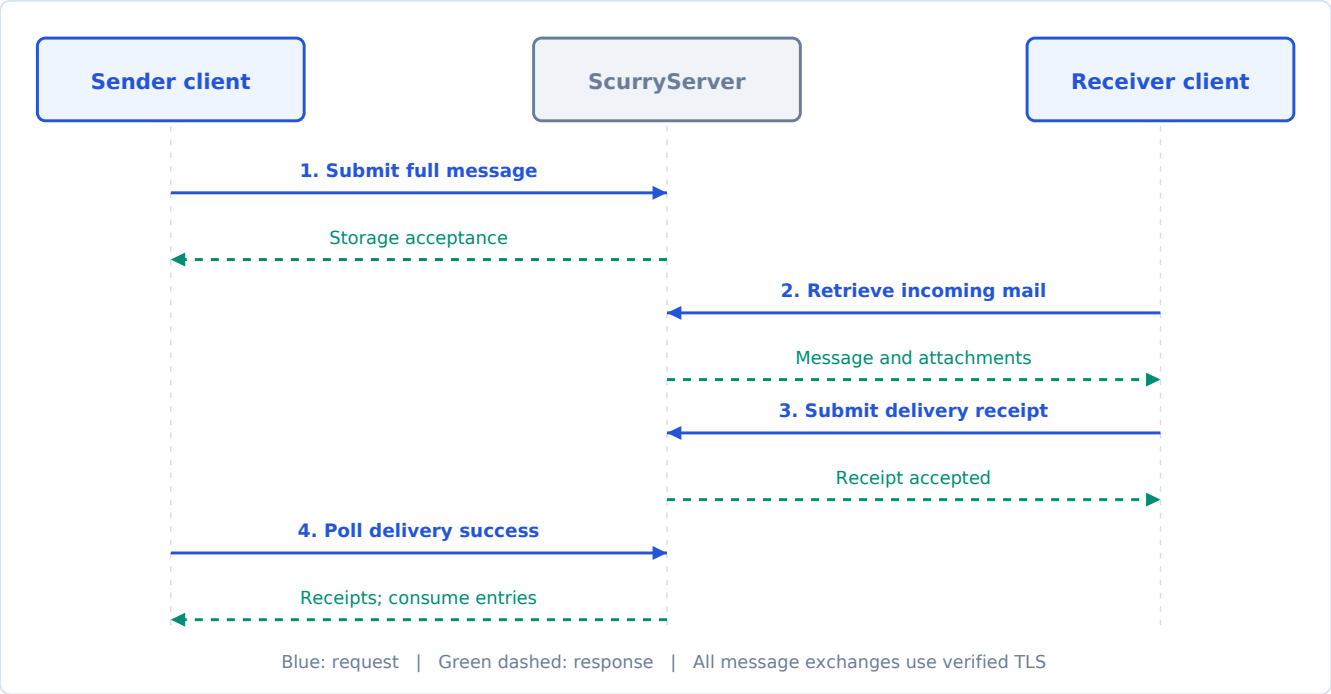


Figure 2. Central mailbox delivery. The receiver retrieves mail, persists it and submits its acknowledgement. Sender polling returns queued receipts. Consumption follows the complete server response write and retains the documented crash window.

Both clients initiate TLS connections to their configured ScurryServer. The sender submits content, the server stores it, and the receiver retrieves it when online. The receiver submits its delivery acknowledgement before the sender polls for success. The central example port is TCP 33338, configurable. Offline storage is a mailbox feature, not a push guarantee.

Central mailboxes on different servers

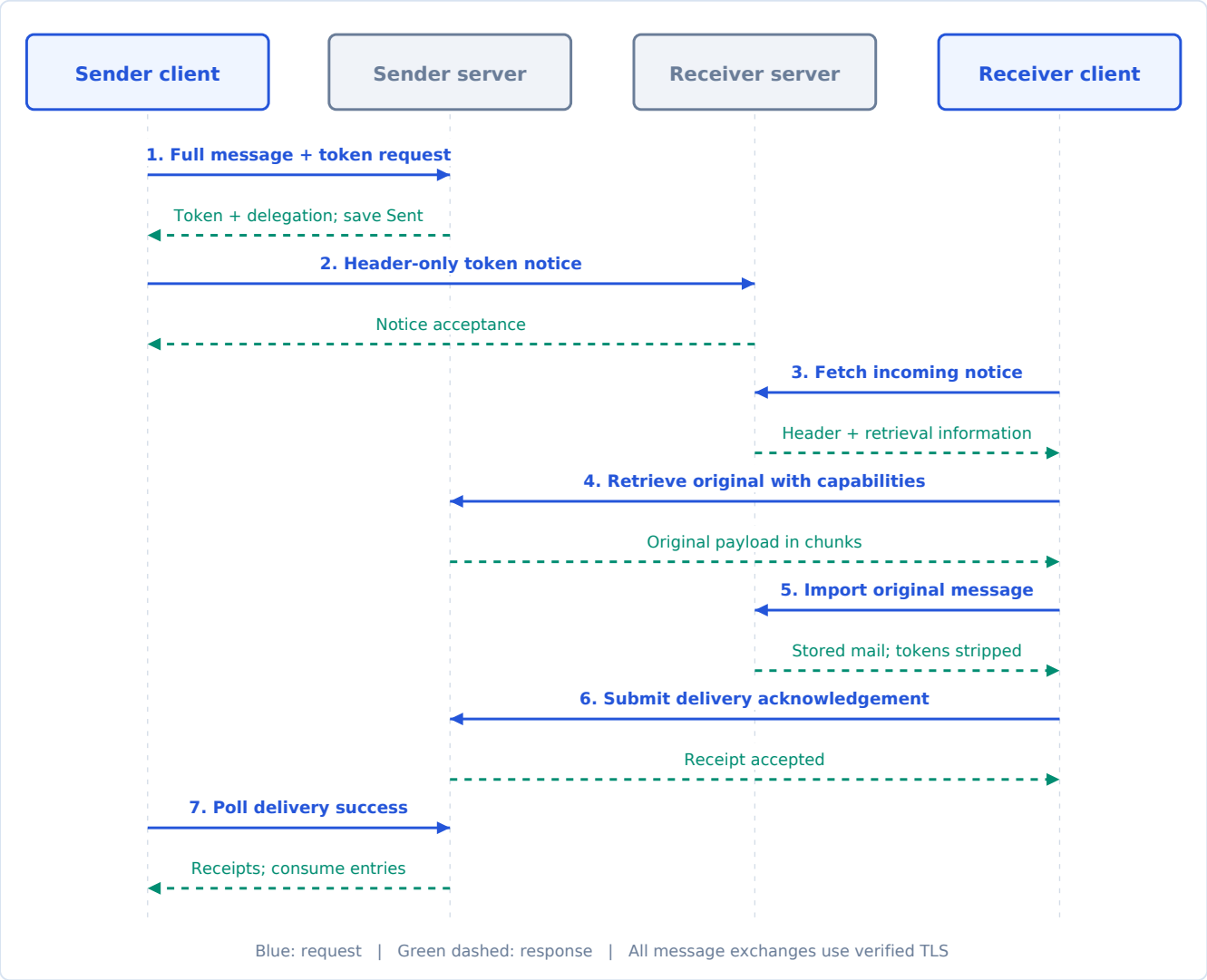


Figure 3. Client-mediated cross-server delivery. All solid and return arrows are client-initiated TLS exchanges. There is no sender-server-to-receiver-server payload trunk. Step 4 uses token-scoped sender credentials; step 5 uses receiver-named account credentials. The receiver durably journals pending import/receipt work before completing steps 5 and 6.

The sender's server archives the original payload. The receiver's server initially stores only a header notice. The receiver's client bridges the two by retrieving from the sender's server and importing into its own mailbox. This is a client-mediated cross-server workflow. The receiver must be able to reach both servers, and the sender must be able to reach the receiver's server for notice submission.

Connection	Purpose	Credential authority
Sender client to sender server	Archive full message and issue token	Sender account
Sender client to receiver server	Submit header-only notice	Remote delegated fields; notice remains unverified
Receiver client to receiver server	Fetch notice	Receiver account, using sender-named request fields

Connection	Purpose	Credential authority
Receiver client to sender server	Retrieve original; submit receipt	Per-delivery delegated credentials and binary capabilities
Receiver client to receiver server	Import original message	Receiver account, using receiver-named fields
Sender client to sender server	Retrieve delivery receipts	Sender account

Transport TLS protects each connection. It does not establish end-to-end encryption against the servers: centralized servers store and process payloads. Encrypted local client files protect local storage under the existing vault model. No fixed latency or server-load advantage is guaranteed by topology alone.

Mandatory envelope

Every request, response, nested message and persisted wire frame uses protocol 1.2. There is no 1.0 negotiation or fallback. Deploy all communicating endpoints together.

The first **header record** is exactly named `Protocol`, with value:

```
Protocol: SCURRY 1.2 UTF-8 UTF-8 /endpoint-string
```

The 64-byte framing preamble is `SCURRY 1.2 UTF-8 <count> <header-bytes> <payload-bytes>`, NUL-padded to 64 bytes. It precedes header records; this is a fixed-slot binary protocol, not an HTTP line protocol. Both preamble and first header are validated. Uppercase `PROTOCOL`, old versions, non-UTF-8 header encodings, misplaced first records and malformed endpoints are rejected. Endpoint handlers also validate their specific allowed paths.

Records retain the 64-byte ASCII name, 8 ASCII decimal capacity, 8 ASCII decimal encoded-length, and fixed-capacity value layout. The encoded-length counts all encoded value bytes, including NUL padding; it is not the unpadded text length. Text capacity counts Unicode scalar values, not UTF-8 bytes. Right-pad to that capacity with NUL scalars, then encode as strict UTF-8. Thus a 64-scalar slot can occupy 64 through 256 bytes. Binary fields always use capacity 256 and encoded-length 256. Header-byte count excludes the preamble and payload; record count includes Header-Count itself. Duplicate header names, compared case-insensitively, are rejected. Text values must not contain embedded NUL, CR or LF. Apostrophes are now valid in header names. Text values use strict UTF-8, scalar-count padding and the existing text capacities. `Sender's-Password`, `Receiver's-Password`, `Protocol`, `Subject`, message identifiers, recipient-list descriptors, attachment descriptors, `Sender-Host`, `Recipient-Host`, and `Sender-Account` have capacity 256; ordinary text headers have capacity 64. At most 128 records and 150000 header bytes are accepted. Payload lengths remain signed 64-bit on the framing layer; configured central request/response limits still apply.

Header	Wire value
`differentiated-endpoint`	Exactly 256 raw bytes; four consecutive 64-byte blocks, central mode only
`Token-Request`	Exactly 256 raw bytes, all zero for issuance
`Random-Id`	Exactly 256 independently random binary bytes
`Random-Password`	Exactly 256 independently random binary bytes
`Sender's-Token`	Exactly 256 server-generated random binary bytes

Binary values are not UTF-8, text, hex or Base64 on the wire. Embedded zero, CR, LF and every other byte are retained. Fixed slot lengths delimit them. Python/C#/Java hold these octets through a Latin-1 carrier string for JSON persistence; their binary setters/getters perform explicit byte conversion. C uses raw byte slots and `scurry_set_binary`. The C server's internal JSON catalog uses hex; that storage encoding never changes the raw binary wire layout.

Credentials

Every central request supplies `Sender's-Login` and `Sender's-Password`. These replace `Login`, `Authorization`, `Sender-Login-Name` and `Sender-Password` on central wire requests. Their existing login/password formats and account-verification rules remain. Values are credentials, not literal labels such as "login name" or "password". Client-certificate authentication is additional transport identity and does not remove these required fields.

Only `/message-retrieval-from-sender-for-storage` uses `Receiver's-Login` **and** `Receiver's-Password` against the receiving account. Sender credentials, if present on that import, do not authorize it and are ignored. Receipt polling, local receipt submission, listing, searching, status and ordinary sends use sender-named credentials, even when the person making that request is the message recipient.

Peer requests also use the sender-named headers. Their password value retains the existing password/nonce/timestamp tuple, per-request freshness and replay checks; central passwords remain direct account credentials over verified TLS.

Central-only differentiated endpoint inputs

Added in source release **1.11.0**, retaining protocol **SCURRY 1.2**. The exact canonical header name is `differentiated-endpoint` (lowercase). It is optional, valid only when `Operating-Mode: Central-Server`, and contains exactly **256 raw binary bytes**. Peer-mode or missing-mode use is rejected by the codecs. There is no text padding, Base64 text or second copy of the data in this wire slot. Its capacity and encoded-length fields are both 256.

Block index	Original byte offsets	Endpoint input
0	0..63	raw[0], exactly 64 bytes
1	64..127	raw[1], exactly 64 bytes
2	128..191	raw[2], exactly 64 bytes
3	192..255	raw[3], exactly 64 bytes

Before invoking the central endpoint handler, dispatch derives two ordered arrays from the same initial header: **raw[4][64]** and **base64[4]**. Each Base64 element encodes only the corresponding raw block using the standard RFC 4648 alphabet, with padding and no line breaks. Every element is 88 ASCII characters and ends in ==. The four strings occupy 352 characters in total; they are application inputs, not a 256-byte on-wire value. Splitting after Base64 encoding the entire 256 bytes would give different results and is forbidden.

The endpoint remains the one in the initial `Protocol` record. Both arrays are handed to that same authenticated operation callback; they do not authorize a different operation or create an HTTP URL, dynamic alias or server-to-server route. A handler may use these differentiation inputs inside its existing authorized operation. Authentication, account isolation, token scope and endpoint allowlists remain in

force. Base64 is a representation, not encryption or authentication.

Library	Endpoint-facing access
Python	`Route.differentiated_endpoint.raw` and `.base64`; `Header.get_differentiated_endpoint()`
C	`scurry_route.differentiated_endpoint.raw[4][64]` and `.base64[4][89]`; final byte of each C string is NUL
C++	Same callback route structure; `header.differentiated_endpoint()` and raw-block setter
C#	`CentralRoute.DifferentiatedEndpoint.Raw` and `.Base64`; `Header.GetDifferentiatedEndpoint()`
Java	`Route.differentiatedEndpoint().raw()` and `.base64()`; `Header.getDifferentiatedEndpoint()`

When the optional header is absent, the endpoint input is absent/null (C: present=0); it is not four synthetic zero blocks. Python, C# and Java also offer setters taking the four raw blocks or four canonical Base64 strings. C/C++ provide raw setters and both derived arrays. Array values are request-local snapshots; handlers must treat them as read-only. Changing a snapshot does not rewrite the message header.

New central composers generate four random 64-byte blocks once per composed message, or accept explicitly supplied raw blocks through their composition API. The same stored envelope is reused on retry. The outer send command carries the same value as its nested original. A supplied outer-only value can be attached to the original; differing outer and inner values are rejected. Send idempotency includes the outer input as well as the original frame/payload.

Per-account persistence is provided by the existing mailbox catalog: original bytes and the original content endpoint remain associated with that account's stored message/Sent copy across connections and server restarts. No separate global routing registry is introduced. Full authenticated retrieval reconstructs both arrays from those saved bytes. Listings redact the raw input; payload backends and client vault formats are unchanged.

Cross-server token issuance, notices, source retrieval and final import preserve this original metadata. Source token binding and receiver notice/import binding compare it. Final import still removes token/random/delegated secrets, but **retains differentiated-endpoint** because it is original message metadata. It must not be confused with Token-Request, Sender's-Token, Random-Id or Random-Password.

Existing central 1.2 frames without this optional header remain valid. Older libraries do not know the new raw slot and must be upgraded before exchanging frames that contain it. The C callback route structure has grown; rebuild C/C++ integrations together rather than mixing old and new ABI objects.

Peer delivery and resume

1. Sender freezes `Message-Sent-Time` at composition and sends `Receipt-Algorithm: SCURRY-PRNG-SHA512-1`. Both fields enter the immutable transfer fingerprint. `Send-Time` refreshes only authentication on retry.
2. Sender queries `/transfer-status` and validates `/transfer-state`, saved byte count and overlap hash. Incomplete payload delivery retains the existing 128-byte overlap rule.
3. Receiver verifies the complete SHA-256 payload and descriptor partition, creates the checksum receipt, and durably commits the message and receipt together.

4. Receiver writes `/delivery-success` plus its bounded UTF-8 receipt body **on the same TLS/TCP connection** that carried the completed message or resumed transfer.
5. A completed status retry returns `/transfer-state` followed by the same persisted `/delivery-success` frame on that retry connection. It does not manufacture a new receive time or duplicate the message.
6. Sender checks IDs, version, mode, frozen epoch, algorithm, expected receiving contact, lengths, original identities and SHA-512 before marking that recipient delivered. Original receipt text is retained in Acknowledgements. A failed check leaves Outbox pending.

Same-server central delivery

`/send-message` carries a nested message frame. The server binds its sender identity to the authenticated account, stores independent recipient copies and Sent archives, and returns the existing message-ID TSV acceptance result. Acceptance is not a delivery checksum receipt.

The recipient downloads and persists the payload, then posts its readable checksum body to `/delivery-acknowledgement` with the server message ID. The server verifies mailbox ownership and the checksum against the stored payload, then queues the receipt for the sender. Repeated submissions are idempotent and cannot recreate a consumed receipt.

The sender polls `/delivery-success`, receives an outer `/central-response` containing nested `/delivery-success` frames, validates them against Sent payloads, and saves them locally. ScurryServer deletes the returned acknowledgement rows after its complete TLS response write succeeds. A minimal digest ledger remains to prevent duplicate submission from resurrecting an acknowledgement. The Python reference server invokes `complete_response()` after a successful stream flush.

This requested destructive fetch is not an end-to-end transactional acknowledgement: a disconnect or sender-process crash after a successful server write but before durable client storage can lose a fetched receipt. Exactly-once consumption would require an additional client-confirmation operation, which is not silently added here.

Different-server centralized flow

1. **Issue:** sender posts `/send-message` to its own server, using its normal sender credentials. The request contains the full nested message, zeroed `Token-Request`, fresh `Random-Id` and `Random-Password`, and reachable `Sender-Host`/`Sender-Port`. The server verifies the payload, saves the original in the sender's Sent folder, generates `Sender's-Token`, and returns the original message metadata and token. Issuance retries with unchanged content and random values return the same issued token.
2. **Delegate:** the issuance response also supplies `Delegated-Login` (32 ASCII hex characters) and `Delegated-Password` (64 ASCII hex characters). These are per-delivery, token-scoped sender credentials for retrieval and receipt submission. The receiver never receives the sender's general account password. All three binary capabilities, immutable metadata, delegated login and delegated password must match the original delivery. Delegation cannot list or read the sender's other mail.
3. **Notify:** sender posts a **zero-body** `/sender-token-notice` to the recipient's server, using those delegated values under `Sender's-Login`/`Sender's-Password`. It includes the token/random fields, full sender/recipient identities, subject, frozen epoch, payload checksum/length, attachment descriptors and sender location. The server stores an account-scoped header-only notice and returns an acceptance acknowledgement. A notice is unverified incoming metadata until the receiving client retrieves and verifies the source payload. Delegated credentials do not authenticate a local destination account.
4. **Retrieve:** on refresh or opening mail, the receiver recognizes `Token-Notice: 1` and contacts the advertised sender host with verified hostname/CA TLS. It sends `/message-retrieval-from-sender`, the

delegated sender credentials, all three binary capabilities and original binding metadata. `Resume-Offset` and `Retrieval-Length` select an exact chunk (maximum 65536 bytes). Each complete chunk is verified for identity/range and saved in the encrypted local partial file; retries restart at the last committed chunk. The final assembled payload must match SHA-256 before import.

5. **Import:** receiver posts `/message-retrieval-from-sender-for-storage` to its own server, with a nested complete original message, `Notice-Id`, and receiver credentials. Neither outer nor nested import contains `Sender's-Token`, `Random-Id` or `Random-Password`. Sender identity, subject, payload and attachment descriptors are preserved. The server validates recipient ownership and notice binding, stores the payload through the configured backend as incoming ScurryMails, and replaces the notice while preserving its public message ID. Retries do not duplicate the imported mail.
6. **Receipt:** receiver persists an exact receipt and pending journal, completes the import, then posts `/delivery-acknowledgement` to the sender's server using the scoped sender credentials and capabilities. That server verifies it against the archived payload and queues it for the sender. Failed submission is retried from the encrypted pending journal, including after restart. The sender retrieves it through `/delivery-success`; successful fetching removes the queued acknowledgement.

The Windows composer's **Recipient server** button opens optional routing settings; the Python composer shows those settings inline. The settings include recipient-server `host:port` and remote recipient-full-name fields. Leave the server field blank for same-server delivery. Use a sender hostname reachable from the recipient's machine. Configure CA trust covering both servers; self-signed test identities require explicit trust and do not justify accepting arbitrary remote certificates. Clients use encrypted flat files; databases remain server-side only.

Deterministic checksum algorithm

The earlier proposal left the PRNG, byte order, rounding tie, packing order, below-range prime behavior and checksum byte scope unspecified. Protocol 1.2 fixes those choices as follows:

1. Freeze a new `Message-Sent-Time` header as the nonnegative original message epoch in UTC seconds. `Send-Time` remains the fresh authentication clock on each attempt. Add `Receipt-Algorithm: SCURRY-PRNG-SHA512-1`. Both new headers enter the transfer fingerprint. This avoids changes to the checksum across authentication refreshes and resumed deliveries.
2. Round the original epoch to the **nearest hour**, with exact half-hours rounded forward: `hour = ((epoch + 1800) // 3600) * 3600`. This differs from truncation. Hash UTF-8 `FullName + LF + Email + LF + decimal(hour)` with MD5. Preserve case, spacing and Unicode as transmitted. Reject CR/LF in names/emails.
3. Read digest bytes 0-1 as an unsigned big-endian value. Clamp to 257-65521, then select the smallest prime at or above the clamped value. The result is strictly greater than 2^8 and less than 2^16. Thus 65535 maps to 65521. The example stride 13 illustrates sampling only; 13 cannot satisfy the requested greater-than-8-bit bound.
4. Read the **second 32-bit word**, digest bytes 4-7, big-endian. Set `seed = (word & 0x0fffffff) | 0x00080000`. Its bit length is 20 through 28 inclusive. This defines the range mapping rather than relying on language-specific PRNG seeding.
5. Common PRNG: 32-bit LCG, `x0=seed`, `xn=(1664525*x(n-1)+1013904223) mod 2^32`. Select the 1-based generated words **x1, xp, x(2p), x(3p), ...**. Affine skip-ahead computes exactly the same words without millions of discarded calls. Do not use Python `random` or .NET `Random`.
6. Retain each word's lowest 29 bits. Append each retained word most-significant-bit first, filling bytes most-significant-bit first. Append **2259 complete words (65511 bits)** and **25 trailing zero bits**. The result is exactly 8192 bytes. Do not append a partial 2260th word.

7. Stream `SHA512(original_payload_bytes || deterministic_suffix)`. Payload means original plaintext body plus all attachment bytes in descriptor order, with no transfer-overlap bytes, no encrypted storage envelope, no TLS bytes, and no wire headers. Header binding remains in the authenticated transfer metadata; this checksum does not bind every header field.

Receipt body

`/delivery-success` carries these eight LF-terminated lines, strict UTF-8, at most 16384 bytes. `Body-Length` equals the frame payload length. The envelope correlates message and delivery IDs, original SHA-256, frozen epoch and receipt algorithm.

```
Checksum-Algorithm: SCURRY-PRNG-SHA512-1
Message-SHA512: <128 lowercase hexadecimal digits>
Sender-Name: <original sender full name>
Receiver-Name: <receiver full name>
Sender-Email: <original sender email>
Receiver-Email: <receiver email>
Sent-Time: <receiver-local ISO date/time with numeric UTC offset> [<zone>]
Received-Time: <receiver-local ISO date/time with numeric UTC offset> [<zone>]
```

Sent-Time represents the original frozen send epoch formatted in the receiver's local zone; it is not the acknowledgement transmission time. Received-Time represents the completion time after full payload verification and durable delivery. Retries replay those original strings. A receipt is scoped to one Delivery-Identifier and receiving identity; multi-recipient sends use distinct delivery identifiers.

A public deterministic suffix is not a signature, authentication secret or replacement for TLS. Receipts bind payload bytes and the documented identity/time derivation; they do not authenticate every arbitrary header field. Preserve a receiver's original timestamp strings when displaying the receipt elsewhere.

Endpoint and response contract

The following paths are SCURRY Protocol values, not HTTP URLs. Outer central replies use `/central-response`; nested content retains its own 1.2 Protocol record. An acceptance response confirms storage or submission, not delivery to a human reader.

Request	Response or effect
Peer `/message` and supported content paths	`/delivery-success` receipt on the completion connection
Peer `/transfer-status`	`/transfer-state`; also saved `/delivery-success` if complete
Central `/send-message`	Acceptance TSV; with Token-Request, issuance metadata and token
`/sender-token-notice`	Notice acceptance and server mailbox identifier
`/message-retrieval-from-sender`	Original message frame containing the requested payload range
`/message-retrieval-from-sender-for-storage`	Receiver-authenticated import result, retaining notice mailbox ID
`/delivery-acknowledgement`	Receipt submission acceptance
Central `/delivery-success`	Nested receipt frames; consume only returned queue entries after complete response write

A resumed source download frame carries a chunk, while original Total-Length and Content-SHA256 describe the complete original payload. Validate the frame payload length and range separately from the original total. Do not compute the final checksum over only the current chunk. Preserve the original content endpoint, body format, attachment offsets and descriptor order when importing.

Readable field examples are projections only:

```
Protocol: SCURRY 1.2 UTF-8 UTF-8 /sender-token-notice
Operating-Mode: Central-Server
Sender's-Login: ASCII-data-00000001-scoped-login
Sender's-Password: ASCII-data-00000002-scoped-password
Sender's-Token: Binary-data-00000001-exactly-256-raw-bytes
Random-Id: Binary-data-00000002-exactly-256-raw-bytes
Random-Password: Binary-data-00000003-exactly-256-raw-bytes
Subject: UTF-data-00000001-original-subject
```

This excerpt is incomplete and not directly executable. Full identity, location, message/delivery identifiers, timestamp, checksum, length and descriptor fields are required by the applicable operation. The literal labels are not credentials or binary encodings. Use the companion SCURRY-1.2-MESSAGE-EXAMPLES.md for every request/response family, including ordinary mailbox operations.

Failure handling and trust boundaries

- Reject malformed framing, unsupported protocol versions, duplicate names, unknown/disallowed endpoints and invalid binary lengths before using a request. Authentication and authorization must be scoped to the actual account or delegated delivery.
- Validate the full payload hash, body/attachment partition and immutable message metadata before committing delivery. Resume authentication freshness may change; original delivery identity and Message-Sent-Time must not change.
- On issuance or notice retry, unchanged delivery identifiers and content return the previous result. Conflicting content is rejected. An import retry preserves one public mailbox ID. Repeated identical receipts are idempotent; changed receipts conflict.
- Do not label a notice as verified incoming content. The destination server cannot independently authenticate the remote sender account from delegated fields alone. Fetch the original from the trusted source service and validate binding before import. The receiving server validates the importing account and notice binding; it does not independently re-fetch the source.
- Delegated retrieval and receipt credentials are bearer capabilities. Possession, including possession by the original sender, can authorize a token receipt. The receipt does not provide independent receiver-signature proof, non-repudiation or evidence that a person read the message.
- Keep account passwords, delegated credentials and tokens out of logs and public listings. Do not forward the sender's general account password to the recipient. Final imported mail must exclude binary capabilities and delegated authentication secrets.
- Serialize destructive receipt polling per sender where possible. Concurrent fetches can observe the same pending receipt before deletion. Client deduplication uses delivery identity and receipt digest. A complete write is the consumption boundary, not proof of durable sender storage; the loss window described above remains.

Deployment controls still requiring explicit design or acceptance

The reviewed release does not establish a complete token expiry/revocation policy, notice abuse quotas, or a restrictive policy for connections to notice-supplied source hosts. Before public deployment, define capability lifetime and revocation, credential/notice retention, rate and storage limits, and source-host/network restrictions. An untrusted notice must not cause unrestricted connections to internal or privileged services. Verified TLS alone does not define which remote hosts are authorized for this workflow.

These are open deployment requirements, not claims that those controls have already been implemented. Token reissue responses and notice storage contain reusable secrets and require protected access, backup and retention policies. The server's payload adapters do not themselves prove encryption at rest.

Validation scope

The earlier architecture-only review checked all three original PDF pages, the current 1.2 design and RFC, the companion exchange examples, Python framing/peer/central/federation/receipt source and native central 1.2 handling. It corrected architectural and field-contract inconsistencies. That review did not rerun application tests. Release 1.11.0 subsequently adds the executed codec, native server, persistence, cross-server and C# checks recorded in VALIDATION-1.11.md. Native Windows GUI/service acceptance remains pending. The earlier baseline evidence remains in VALIDATION-1.10.md. A document correction is not evidence that the open deployment controls above have been implemented.

Compatibility and deployment

1.0 is deprecated and rejected by released 1.2 codecs. Historical 1.0 documents remain for migration reference. Stop old listeners and upgrade server/libraries/clients together. Existing encrypted vault keys and account-password formats remain; do not replace vault files. Old queued 1.0 frames require explicit review/recomposition as 1.2 before sending. Already stored server frames using old protocol headers require a deliberate offline migration; there is no silent wire fallback or automatic rewrite of old mail. Back up the existing catalog and payload store before deployment.

The new native metadata tables are created automatically on first use; payloads still use SQLite, ODBC SQL, CouchDB or configured Cassandra content adapters. New SQLite reference tables are created on initialization. Native Windows GUI/service runtime and the Python 3.15 target runtime require platform acceptance; see the current validation report for executed checks.

Readable samples for every endpoint family | unnumbered RFC | validation.